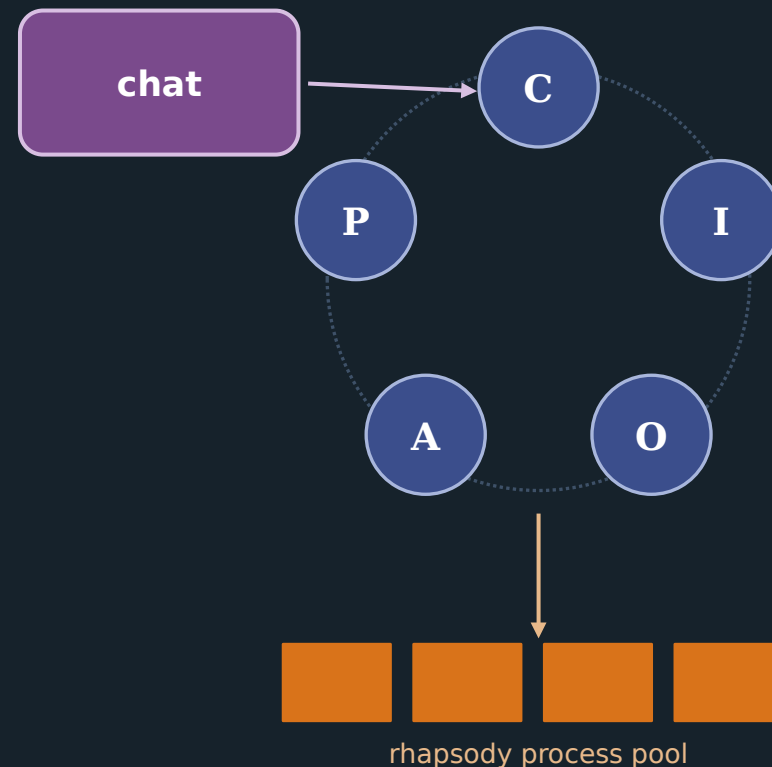


DESIGNAGENT

A chat agent that runs real redesign campaigns

A LangGraph loop whose long work leaves the process —
and a chat that keeps talking while it does

Worked example, measured: 1OIL (P22088, 320 aa) · 2 rounds · 11 designs ·
pLDDT 81.66 → 81.79



Lab code walk · main @ e8467e6 · ~7.6k lines backend, 1291 lines of tests

WHAT IT DOES

One prompt, a whole campaign



What came back, for real (campaign s-tutp87hw)

- 1OIL — P22088, Burkholderia cepacia lipase, 320 residues, resolved from the prompt alone
- lead design G147A at pLDDT 81.79, RMSD 3.75 Å to the crystal structure
- 11 designs scored on 10 metrics; 114 score rows written
- a Markdown summary, a .docx, a sortable ensemble table, and a Mol* view of the lead

And what that is not

- The variants are heuristic single-point proposals, not ProteinMPNN samples — no HPC endpoint was attached.
- A pLDDT move from 81.66 to 81.79 is inside the noise. The loop is real; the science is a demo.
- This run had no ANTHROPIC_API_KEY. Every node took its deterministic path.

The point of the demo: a prompt in a browser produced a provenance graph, a scored ensemble and a training-ready Parquet set, with no human in the middle and nothing stubbed.

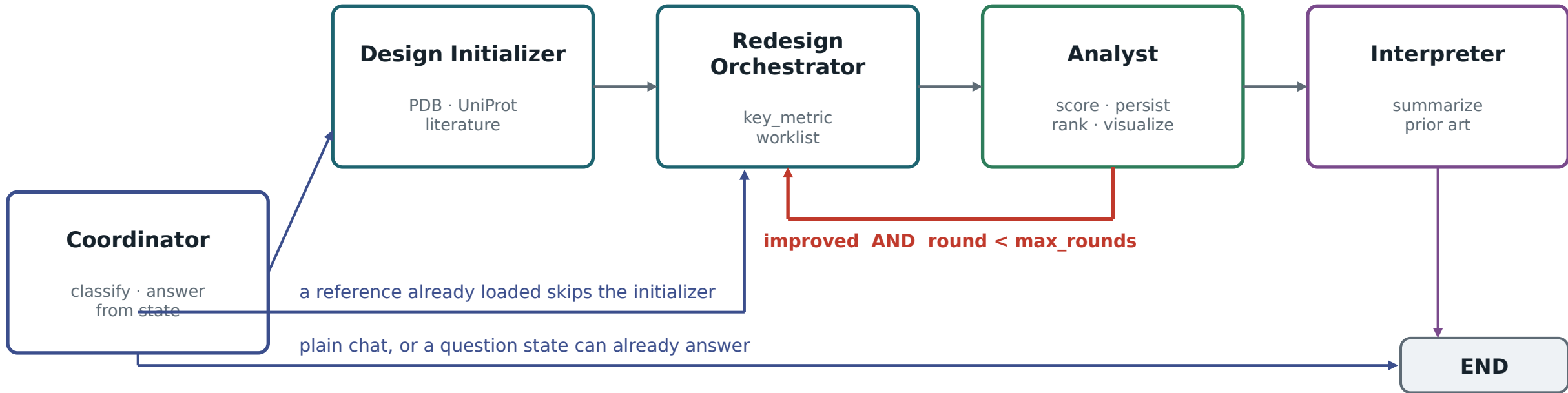
One constraint sets the whole design

A chat interface must answer in milliseconds. A protein fold takes tens of seconds. An HPC job takes hours, and may sit in a queue first.

consequence	what it means in the code	where
Tasks return futures, never results	Every interface's submit() places the work and returns a handle. Nothing in the graph awaits a task at submission time.	tasks/base.py:146
Task duration is not the interface's business	The same handle covers a 40 ms REST call and a queued batch job. Only the capability flags differ.	tasks/base.py:62
Status is a stream, not a return value	Nodes emit progress over LangGraph's custom channel; the manager fans task events onto the same SSE stream.	graph/deps.py · app.py:78
State holds references, not payloads	Coordinates go to a content-addressed blob and travel as a path, because every checkpoint is serialized.	orchestrator.py:249
Every layer has a floor	No key, no endpoint, no graph DB: each degrades to something that still answers, and says so.	runtime.py · analyst.py:154

Nothing below this slide is a free choice. If you drop the constraint — if the agent may block — most of the Task Interface layer stops paying for itself and you would call the tools directly.

Five nodes, and the nodes do their own routing



```

destinations = {
  "coordinator": ("initializer", "orchestrator",
                 "analyst", "interpreter", END),
  "initializer": ("orchestrator", END),
  "orchestrator": ("analyst", "interpreter", END),
  "analyst": ("orchestrator", "interpreter", END),
  "interpreter": (END,),
}
builder.add_edge(START, "coordinator") # the only static edge

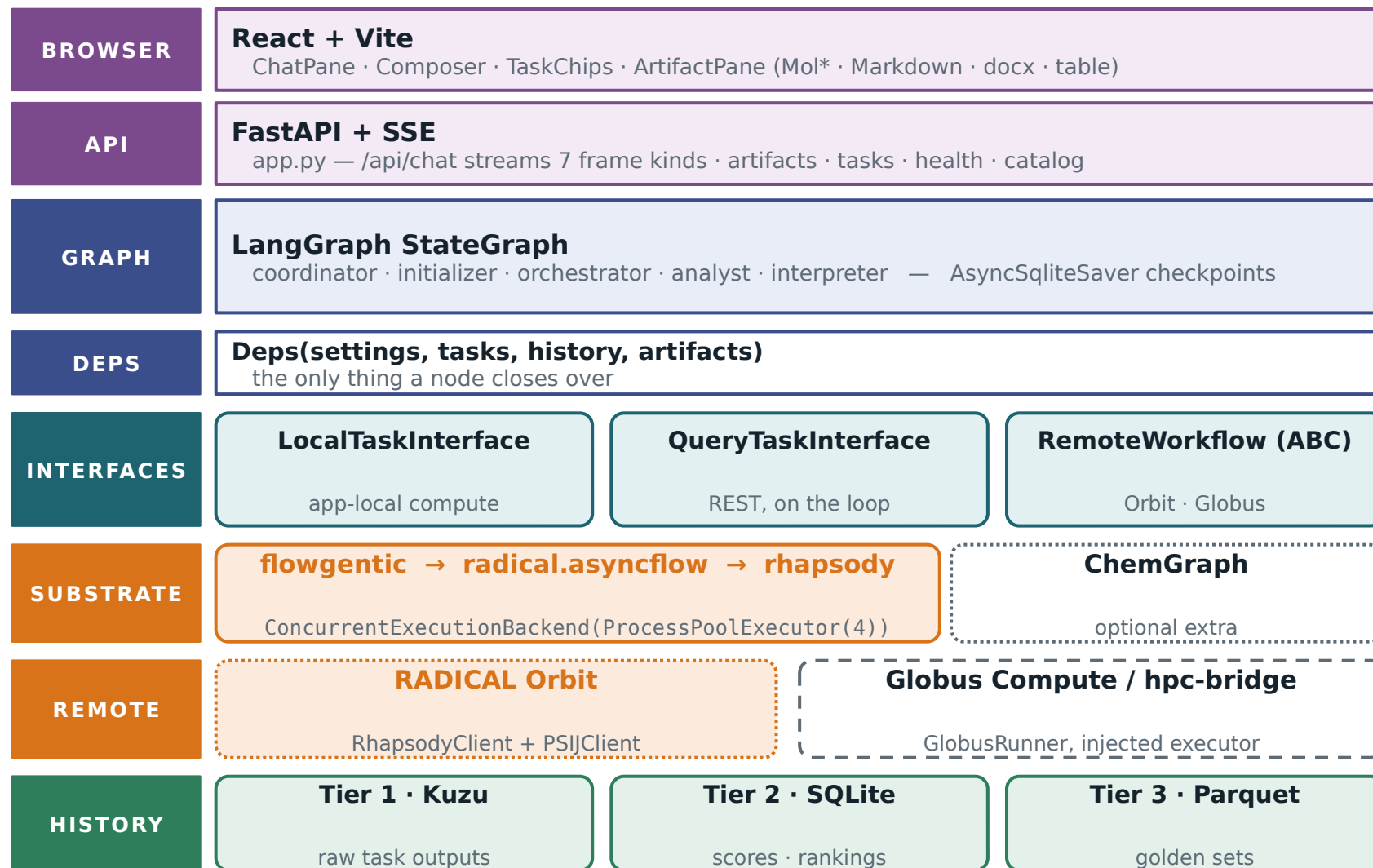
```

graph/build.py:99–114 · VERBATIM

Why no conditional edges

- A node returns `Command(goto=..., update=...)`: the decision and the state write are one atomic return.
- `destinations=` is a declaration for validation and the drawn graph, not control flow.

Architecture, and where the RADICAL stack sits



The one rule

- A node reaches the outside only through Deps — never an interface or a store directly.
- That is what makes 93 of 99 tests run with no network, no pool and no endpoint.
- Convention, not an enforced check.

Where it leaves the process

- LocalTaskInterface is the only band that crosses into asyncflow.
- Everything above it is ordinary async Python, and stubbable.

- built, runs end to end
- built and tested, never run for real
- - - designed for, not implemented
- RADICAL component**

State, its reducers, and what must never enter it

```
class DesignState(TypedDict, total=False):
    messages: Annotated[list, add_messages]

    reference_design: Annotated[ReferenceDesign, replace]
    key_metric: Annotated[KeyMetric, replace]
    worklist: Annotated[list[WorkItem], merge_worklist]
    lead_design: Annotated[Design, replace]
    ensemble: Annotated[list[Design], replace]
    molecular_visualization: Annotated[MolVisualization, replace]
    artifacts: Annotated[list[ArtifactRef], merge_artifacts]
    design_summary: Annotated[str, replace]

    # control
    intent: round · session_id · goal · status
    warnings: Annotated[list[str], merge_warnings]
    target_hints: requested_mutations · pending_results
```

graph/state.py:168–196 · TRIMMED

Four reducers, each for a reason

- merge_artifacts — append, dedupe by id, last write wins: the analyst replaces a viz each round.
- merge_worklist — upsert by id, so status can be updated in place.
- merge_warnings — accumulate, dedupe, keep 20. Survives the turn; status does not.

Reducers are the only place concurrency shows up in state. Nodes run sequentially here, but artifacts and warnings arrive from different nodes in the same turn, and both must accumulate rather than clobber.

The finding: a checkpoint is serialized on every turn

First build put fold coordinates in pending_results. Six PDB files is ~320 KB of checkpoint per turn, and it grows with the ensemble. The orchestrator now writes coordinates to a content-addressed blob and passes a path:

```
summary["structure_path"] = deps.history.write_blob(
    structure, suffix=".pdb", prefix=summary["design_id"])
```

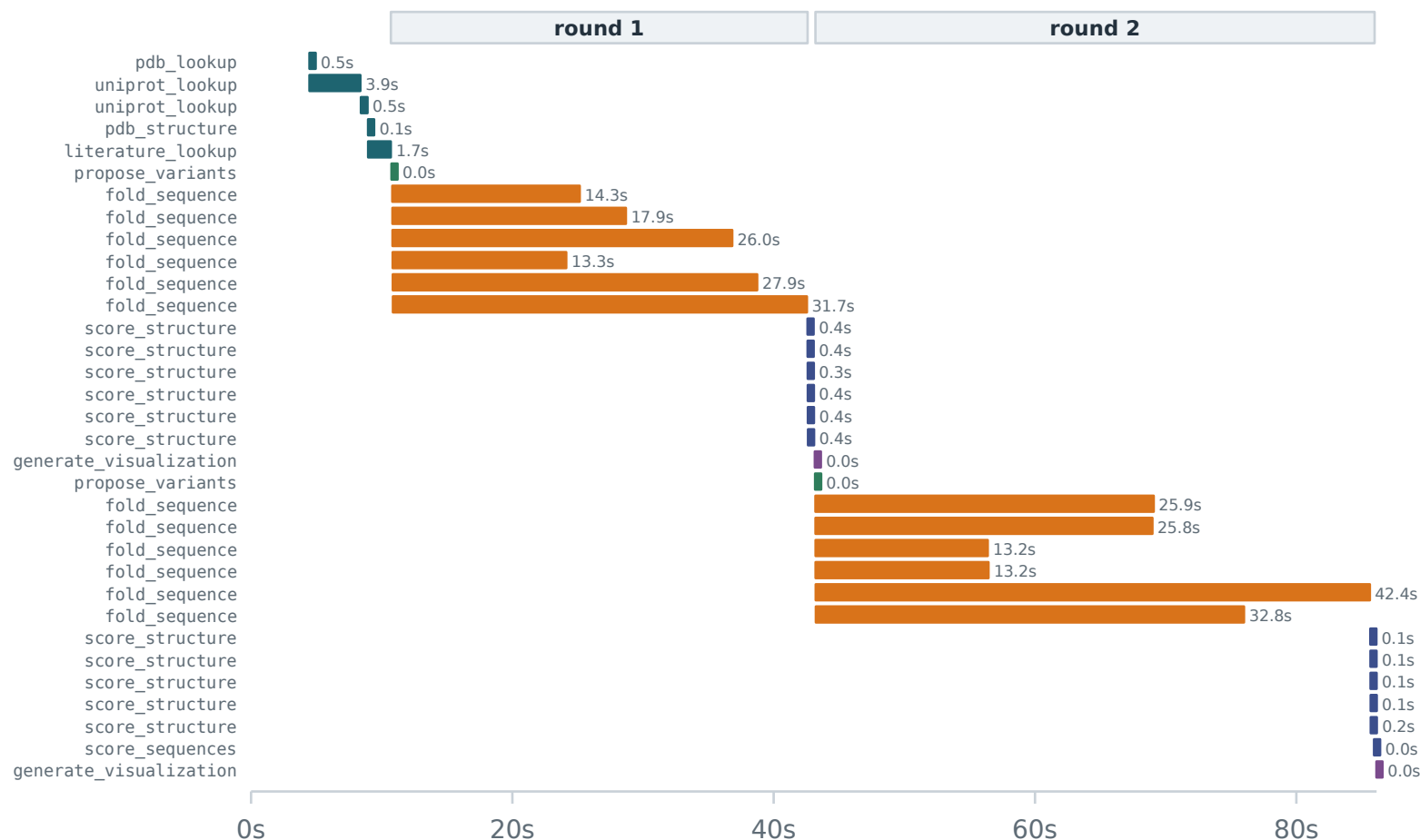
orchestrator.py:249–253 · TRIMMED

```
def test_structures_are_not_carried_in_state(...):
    blob = json.dumps(jsonable(out), default=str)
    assert "ATOM " not in blob
```

tests/test_graph.py · TRIMMED

Pinned by a test, because the regression is invisible: everything still works, just slower every turn.

One campaign, measured: 37 tasks in 146 seconds



Read the fold rows

- Six submitted in the same instant, finishing 13.3 s to 31.7 s later — out of order, as they land.
- Round 2 ran 13.2 s to 42.4 s. flowgentic's 30 s default timeout would have killed half of them.

Read the lookup rows

- pdb_lookup and uniprot_lookup go out together; a cross-reference follow-up resolves second.
- Then structure download and literature, also concurrent.

And one real failure

- ESM Atlas dropped one of round 2's six folds: "s-tutp87hw-r2-5: ESM Atlas returned no structure".
- That design survived as a sequence-only entry with 4 metrics; the round scored the other 5 and the user got a warning.

★ The Task Interface contract

```
class TaskInterface(ABC):
    name: str = "base"
    capabilities: Capabilities = Capabilities()

    @abstractmethod
    async def submit(self, spec: TaskSpec) -> TaskHandle:
        """Place the task and return immediately."""

    async def status(self, handle) -> TaskState
    async def logs(self, handle, offset=0) -> LogChunk
    async def result(self, handle) -> Any
    async def cancel(self, handle) -> bool
    async def close(self) -> None
```

tasks/base.py:135–173 · TRIMMED

```
@dataclass(frozen=True)
class Capabilities:
    supports_cancel: bool = False
    supports_log_stream: bool = False
    supports_push_events: bool = False
    supports_staging: bool = False
```

tasks/base.py:62–67 · VERBATIM

interface	what it is	cancel	logs	push	stage
local	app-local compute, rhapsody pool	✓	—	—	—
query	REST retrieval, on the loop	✓	—	—	—
orbit	Rhapsody tasks + PSI/J jobs	✓	✓	✓	✓
globus	Globus Compute (hpc-bridge)	part	—	—	—

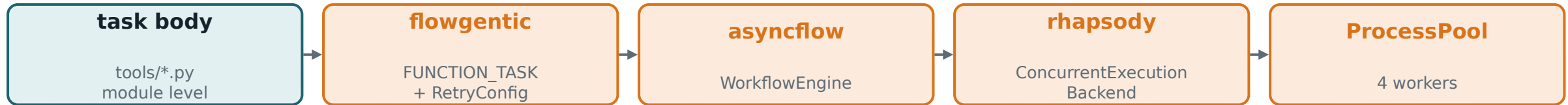
The flags are the honest part. An interface that cannot tail logs says so, and the manager simply does not start a drain for it — no caller has to know which backend it got.

```
try:
    handle = await interface.submit(spec)
except Exception as exc:
    # Surface the failure as a settled handle so callers
    # need no separate error path.
    future = loop.create_future()
    future.set_exception(exc)
    handle = TaskHandle(..., state=TaskState.FAILED,
                        error=str(exc))
```

tasks/manager.py:99–115 · TRIMMED

The invariant: a caller holds a handle whose future resolves. Not sometimes a handle and sometimes an exception. A rejected submission is a handle that has already failed — which is why gather() never raises and the round survives a dead endpoint.

★ Through flowgentic to a rhapsody process pool



```

retry = RetryConfig(
    # Folding or an HPC job can take far longer
    # than flowgentic's 30s default.
    timeout_sec=None,
    max_attempts=1,
    retryable_exceptions=(ConnectionError, OSError),
)
def wrap(body):
    return integration.execution_wrappers.asyncflow(
        body, flow_type=AsyncFlowType.FUNCTION_TASK,
        backend="compute", retry=retry)
  
```

runtime.py:76–89 · TRIMMED

```

async def submit(self, spec: TaskSpec) -> TaskHandle:
    fn = self._resolve(spec.name)
    # create_task so submission returns immediately: the
    # flowgentic wrapper for FUNCTION_TASK is a coroutine
    # function, not a future factory.
    future = asyncio.ensure_future(_call(fn, spec))
    return self._handle(spec, future)
  
```

tasks/local.py:60–65 · VERBATIM

Three constraints the pool imposes

- Task bodies live at module level in tools/ — no closures over clients, or they will not pickle.
- Clients are built inside the body; a parent's cannot cross into a worker.
- The entry point needs a `__main__` guard, or uvicorn re-imports under fork.

Finding 1 — flowgentic hard-imports aiohttp

```

# Try to include aiohttp timeouts if present
try:
    import aiohttp # type: ignore
    ex.extend([aiohttp.ClientConnectionError,
               aiohttp.ServerTimeoutError])
except Exception:
    raise
  
```

refcodes/flowgentic/.../fault_tolerance.py:73–84 · VERBATIM

The comment says "if present"; the code says "or die". Same shape for httpx at :60–72. Reached whenever `retryable_exceptions` is left at its default, which is `()`.

★ RADICAL Orbit: two clients, two concurrency bridges

EndpointRuntime(broker_url, cert, name, token)

RhapsodyClient

function + executable tasks
register_session · submit_tasks
wait_tasks · get_task · cancel_task

pushes task_status events

PSIJClient

batch jobs
submit_job · cancel_job
get_job_status(id, stdout_offset,
stderr_offset)

the only log tail that exists

Two threading problems, two one-liners

- Every Orbit client method is synchronous and blocking → each of 23 call sites goes through `asyncio.to_thread`.
- Push callbacks arrive on Orbit's listener thread → `_dispatch` hops to our loop with `call_soon_threadsafe`.

```
def _dispatch(self, fn, *args) -> None:
    """Hop from Orbit's listener thread onto our event loop."""
    if self._loop is None or self._loop.is_closed():
        return
    self._loop.call_soon_threadsafe(fn, *args)
```

tasks/hpc/orbit.py:261–265 · VERBATIM

Findings 4 and 5 — what a terminal event does not carry

```
# The terminal event carries state and exit_code but not
# stdout, so the full record is fetched before the future
# is resolved.
if not data.get("stdout") and self._rhapsody is not None:
    info = await asyncio.to_thread(
        self._rhapsody.get_task, handle.id)
    # The event wins on state; the fetch fills in the output.
    merged = {**info, **{k: v for k, v in data.items()
                        if v not in (None, "")}}
```

tasks/hpc/orbit.py:286–299 · TRIMMED

```
if state is TaskState.FAILED and not error:
    error = (f"exit code {exit_code}" if exit_code not in (None, 0)
            else (data.get("stderr")
                  or handle.log_tail[-500:] or "job failed"))
```

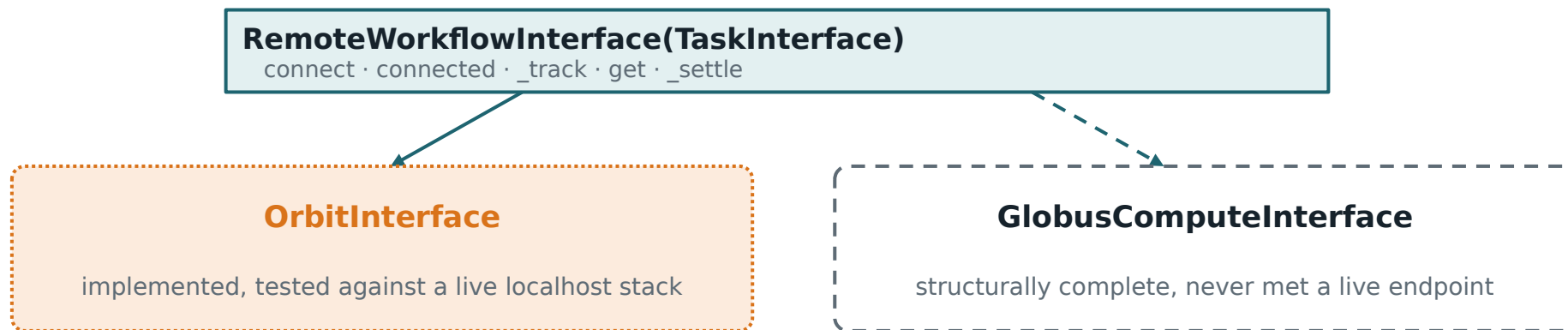
tasks/hpc/orbit.py:353–357 · TRIMMED

A completed task's result came back empty until the re-fetch. A job that exited 3 reported no reason at all — only the code — so FAILED always synthesises an explanation.

Proven, and only this far

- 6 tests against a real localhost broker + endpoint: push states, incremental log tailing by offset, a failing job, and cancelling a running one.
- Never run against a scheduler. That is the next real step.

Designed for two, implemented for one



What the abstraction cost

- Almost nothing: connect/_track/_settle plus drain_logs is the whole shared surface, and it is 102 lines.
- The interesting work is backend-specific by nature — Orbit's push callbacks have no Globus analogue.
- So the ABC's job is to make the differences declarable rather than to hide them.

The claim I will defend: the capability flags are more valuable than the shared base class. They are what let the manager and the UI behave correctly against a backend they were not written for.

```

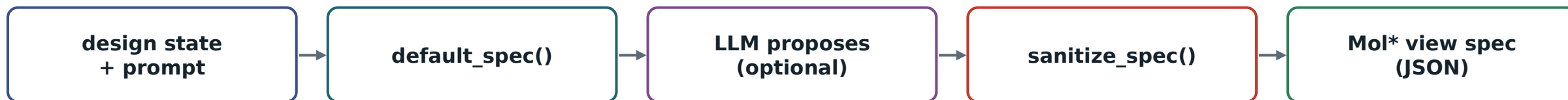
capabilities = Capabilities(
    supports_cancel=False,      # only before start; see cancel()
    supports_log_stream=False,  # final stdout only
    supports_push_events=False, # the future resolves; no
                                # intermediate states
    supports_staging=False,     # shared FS or Globus Transfer
)
  
```

tasks/hpc/globus.py:31–36 · VERBATIM

Testable with no endpoint

- GlobusRunner's executor_factory is injectable, so the suite passes a plain executor taking a callable.
- argv is a list run with no shell on that path; the real ShellFunction path uses shlex.join because its API takes a string. A test proves a metacharacter stays data.

Local Task Agents: a spec, not generated code



Why a spec and not generated JavaScript

- The brief said the agent "codes a browser-based visualization". Shipping LLM-written JS into the viewer is an injection surface for a prompt.
- So the agent emits a constrained JSON spec — structures, highlights, representation, colours — and the React component is the only thing that touches Mol*.
- `sanitize_spec` repairs or drops every field. A bad generation degrades to a plain cartoon, never a broken pane.
- Same expressive range for what a reviewer actually asks for; no execution.

```
def sanitize_spec(spec: Any, *, fallback: dict) -> dict:
    """Validate an LLM-proposed spec, repairing or dropping
    bad fields.
```

```
    Anything unexpected falls back rather than reaching the
    viewer, so a bad generation degrades to a plain cartoon
    instead of a broken pane.
```

```
    """
    if not isinstance(spec, dict):
        return fallback
```

tools/molviz_agent.py:143–150 · TRIMMED

agent	interface	what it does	status
Molecular Visualization Generator	local	Mol* view spec keyed from state + prompt	runs; drew both leads in the measured campaign
Cheminformatics Agent (ChemGraph)	local	ChemGraph single_agent workflow in a pool worker	wired, optional extra, exercised by tests only
ESMFold	local → hpc	ESM Atlas API under 400 aa, else a job	12 real predictions on disk
ProteinMPNN	hpc	PSI/J job spec + FASTA parser	no endpoint → heuristic proposer instead

Design History: three tiers, and what is actually in them

Tier 1 — raw task outputs	Tier 2 — scores, rankings, analysis	Tier 3 — golden sets
Kuzu, embedded graph DB	SQLite	Parquet + manifest
• 6 node tables · 8 rel tables • Campaign 1 · Reference 1 · Design 12 • Task 37 · Output 37 · Structure 11 • properties columns hold JSON, so a new task needs no migration • single writer behind an RLock	• 114 score rows over 10 metrics • 17 ranking rows (round 1: 6, round 2: 11 cumulative) • 2 round analyses, with the failures list • best_designs() can exclude the current campaign — that is how the • interpreter finds prior art without finding itself	• 11 rows, one metric_<name> column per metric seen • rules: metric=plddt, direction=max, top_k=12, • dedupe_sequences=true → 12 designs became 11 rows • the manifest records the rules, so a set is reproducible • staged for ML training, not for reading

```
def record_task_result(self, campaign_id, task_id, *, name, interface, state,
                        result=None, error="", designs=None, round_no=0,
                        reference_id=None, blob=None) -> dict:
    """Write one task's outcome across tiers 1 and 2.

    `blob` is an optional (data, suffix) pair for bulky raw output.
    Returns {"output_id", "blob_path"} for the caller to reference.
    """
```

One facade, three stores

- DesignHistory is the only thing nodes see; tiers 1 and 2 are written together or not at all.
- Bulky output goes to a content-addressed blob (sha256[:16]) — 24 files, 5.1 MB on disk.

lake/store.py:87–106 · TRIMMED

Worth noticing: only tier 1 needs a running process to read. Tiers 2 and 3 are a SQLite file and a Parquet file, so the figures on this slide were produced by a script that opened them directly while the server held the Kuzu lock.

Degradation is a feature, and it is tested

when	what happens	evidence
No ANTHROPIC_API_KEY	every node has a deterministic rule-based path	the measured campaign ran this way
No Orbit endpoint	interface_for falls hpc → local and logs it	tests/test_tasks.py
ChemGraph not installed	the task reports unavailable; nothing else changes	chemgraph_available()
ESM Atlas drops a request	design survives sequence-only; round scores the rest	really happened: s-tutp87hw-r2-5
Tier 1/2 write fails	warn, rank in memory, tell the user in the reply	test_lake_write_failure_does_not_lose_the_round
No SQLite checkpointer	InMemorySaver, with a note on /api/health	runtime.py:172

```
# A storage failure must not lose the user's round: the designs are
# already in state, so we log, warn, and carry on.
try:
    ranked = deps.history.rank_round(campaign_id, round_no, ...)
except Exception as exc:
    storage_error = storage_error or str(exc)
    log.warning("persisting the ranking failed: %s", exc)
    ranked = _rank_in_memory(combined, metric_name, direction)
...
if storage_error:
    update["warnings"] = [f"Design History write failed: {storage_error}"]
```

graph/nodes/analyst.py:155–219 · TRIMMED

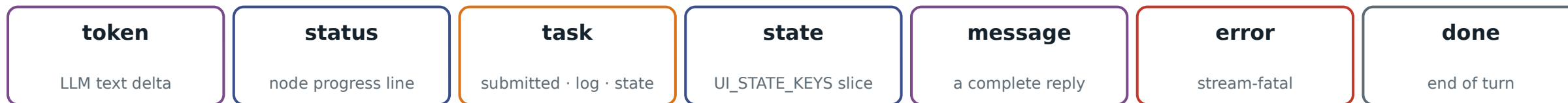
How the user finds out

- The interpreter appends a "Caveats from this run" section to its reply, from the warnings channel.

Where this came from

- A readonly-SQLite error during a live run. My own fault — but it exposed unguarded writes.

The web app: seven frame kinds and one viewer



POST /api/chat returns text/event-stream. One `asyncio.Queue` merges the graph's own astream frames with TaskManager events, so task progress and node status share one ordered stream.

```
let buffer = "";
while (true) {
  const { done, value } = await reader.read();
  if (done) break;
  buffer += decoder.decode(value, { stream: true });
  let split;
  while ((split = buffer.indexOf("\n\n")) !== -1) {
    const raw = buffer.slice(0, split);
    buffer = buffer.slice(split + 2);
    ... yield JSON.parse(line.slice(6)) as Frame;
  }
}
```

A chunk boundary can land mid-frame, so partial frames are held in a buffer until a blank line. A malformed frame is skipped, never fatal.

Confirmed rendering in a browser on 2026-10-01. It was built without one — the dev server, the proxy, a real artifact through it and the CDN assets were all that could be checked at the time.

The Mol* viewer

- rcsb-molstar 2.14.7 UMD from jsDelivr, loader cached on `window.__molstarLoader` so one fetch serves every mount.
- One viewer per mount; the spec is re-applied on change; `ResizeObserver` → `handleResize`.

Two things that bit

- `createComponent` takes no colour, so colours go through `pluginCall` → `updateRepresentationsTheme`.
- Coordinates are inlined as a json artifact by the analyst, so the viewer needs no second authenticated fetch.

HOW TO RUN IT

Running it, and what each test tier proves

```
$ ./scripts/setup.sh           # builds .venv, then verifies it
$ ./scripts/setup.sh --check   # verify only, install nothing
$ python -m designagent --reload # :8000
$ cd frontend && npm install && npm run dev # :5173, proxies /api

$ pytest -q           # 93 tests, no network
$ pytest -q -m live    # 6 tests, starts a real broker
```

VERBATIM

Two things to know

- No key is required: every layer notes what it could not do on /api/health and keeps going.
- uv sync cannot work here: the three middleware packages are local editable installs from refcodes/, and flowgentic pins two of its own deps to git URLs. setup.sh encodes that, and --check fails loudly instead of letting an import die.

file	n	what it actually proves
tests/test_lake.py	11	all three tiers against a tmp_path lake: provenance, upserts, ranking, curation rules
tests/test_tasks.py	25	the contract: futures resolve, failures settle, capability flags honoured, Globus with an injected executor
tests/test_graph.py	—	18 parametrized classifier cases, a full loop end to end, artifacts, lake-write failure, and the no-structures-in-state guard
tests/test_api.py	12	SSE frames, artifact serving, task cancel returning 409 with a reason, health
pytest -m live	6	a real broker + endpoint as subprocesses: push states, offset log tailing, a failing job, cancelling a running one

The split that matters: 93 tests need no network, no pool and no endpoint, because nodes only reach through Deps. The 6 that do are marked `live`, deselected by default, and start their own broker.

What is real, and what is not

Runs end to end against something real

The LangGraph loop, all five nodes

two rounds, routed by the improvement test, on a real target

Query interface

RCSB, UniProt, Europe PMC — live, with cross-referencing both directions

Local interface on the rhapsody pool

12 real ESMFold predictions, 6 at a time, reaped out of order

All three lake tiers

114 scores, 37 task nodes, an 11-row Parquet set

Artifacts

Markdown, .docx, a sortable table, two Mol* specs — all on disk

Streaming

11 status lines plus task chips over one SSE stream

The Mol* artifact pane

confirmed rendering in a browser, 2026-10-01

Built, but has never run for real

ProteinMPNN

a job spec and a parser. No endpoint → a heuristic proposer, labelled as such

The Globus adapter

structurally complete, tested with an injected executor, never met a live endpoint

Orbit

works — against a localhost broker only. Never seen a scheduler or a queue

ChemGraph

wired behind the interface; exercised by tests, never by a campaign

wrap_nodes

ships off, against the approved design. Next slide

The single most load-bearing caveat: no HPC endpoint has ever run a task for this agent. The remote interface is proven against a localhost broker, which proves the client path and nothing about a scheduler.

Six reproducibles, and one question

1 **flowgentic hard-imports aiohttp — and httpx**

fault_tolerance.py:71-72 and :83-84 are except Exception: raise where the comment says "if present". Both become hard requirements. Fires whenever retryable_exceptions is left at its default ().

2 **RetryConfig defaults cancel long tasks silently**

30 s per attempt, 3 attempts. Measured against this repo's own folds: 13-42 s, so roughly half of round 2 would be cancelled and re-run. Long work must pass timeout_sec=None.

3 **Orbit's terminal task event omits stdout**

A completed task resolves with an empty result until get_task is called again. Worked around in _finish_task_enriched.

4 **A FAILED Orbit job carries no reason**

Only a non-zero exit code reaches the client. We synthesise an explanation from exit code, then stderr, then the log tail.

5 **The broker has no HTTP topology route**

/topology → 307 → 404, read as a plugin name. Readiness has to come from the client's rt.topology(), which propagates asynchronously.

6 **--no-auth still requires cert and key**

It disables ingress auth only; the broker always serves TLS and refuses to start without a pair. Worth one line in the docs.

The question: is flowgentic's EXECUTION_BLOCK meant to preserve the caller's context?

A node wrapped as one runs on asyncflow's loop, outside LangGraph's runnable context. get_stream_writer() raises "Called get_config outside of a runnable context" and every custom event is dropped silently — the graph completes, the chat just goes quiet. If yes, it is a bug and wrap_nodes=True becomes our default. If no, flowgentic's node wrapping and LangGraph's streaming are mutually exclusive, and that belongs in the README.

The streaming experiment, in full

What was tried

- The approved design: wrap every node as `AsyncFlowType.EXECUTION_BLOCK` with `RetryConfig(timeout_sec=None, max_attempts=1)`, same as tasks.
- Symptom: the graph completed correctly and the chat showed no status lines at all. No error surfaced anywhere.
- Isolation: one plain node and one wrapped node, each calling `get_stream_writer()` and emitting one event.
- Plain node: writer obtained, event received by `astream(stream_mode=["custom"])`.
- Wrapped node: `RuntimeError "Called get_config outside of a runnable context"`; zero events.

Why

- flowgentic's `EXECUTION_BLOCK` submits the body to the `asyncflow WorkflowEngine`, which runs it on its own loop.
- `LangGraph`'s `get_stream_writer()` reads the runnable context, which is a `contextvar` set by the `LangGraph` executor around the node call.
- Crossing to another loop loses the `contextvar`. The writer cannot be found, and `LangGraph`'s own helper degrades to a no-op rather than raising — which is why it is silent.
- This is not a flowgentic bug unless `EXECUTION_BLOCK` intends to propagate context. Hence the question, not the accusation.

```
def wrap_node(integration, fn):
    """Wrap a node body as a flowgentic EXECUTION_BLOCK, if enabled."""
    if integration is None:
        return fn
    return integration.execution_wrappers.asyncflow(
        fn, flow_type=AsyncFlowType.EXECUTION_BLOCK, retry=node_retry_config())

# build_graph(): integration is set to None unless settings.wrap_nodes
```

graph/build.py:38–87 · TRIMMED

Verified after the change: 11 status lines stream for one redesign turn.

What ships

- `wrap_nodes=False`, the code path kept and documented.
- Tasks are still wrapped — `FUNCTION_TASK` runs task bodies, which never touch the stream writer.
- So the pool is genuinely used; only node bodies stayed in-process.

Standing up a local Orbit stack

```
# broker: --no-auth disables INGRESS AUTH ONLY. TLS is always served,
# and it refuses to start without a cert and key.
openssl req -x509 -newkey rsa:2048 -nodes -keyout key.pem -out cert.pem \
  -days 3 -subj /CN=127.0.0.1 \
  -addext subjectAltName=IP:127.0.0.1,DNS:localhost      # SAN must cover 127.0.0.1
chmod 600 key.pem                                     # refuses a looser mode

radical-orbit-broker.py --no-auth --host 127.0.0.1 --port N \
  --cert cert.pem --key key.pem

RADICAL_ORBIT_RHAPSODY_BACKEND=concurrent \
radical-orbit-endpoint.py --name local -p rhapsody,psij \
  --url https://127.0.0.1:N --cert cert.pem
```

tasks/hpc/local_orbit.py · EDITED for the slide

Readiness

- There is no HTTP topology route — /topology reads as a plugin name and 404s.
- The stack greps the endpoint's own log for registered as '<name>'.

Then the client decides

- OrbitInterface.connect() polls rt.topology() until the endpoint appears, bounded by connect_timeout — topology propagates asynchronously.

Why subprocesses, not EmbeddedBroker

- The embedded broker expects operator-placed credentials in ~/.radical/orbit, which a test must not create or touch.
- Subprocesses with a throwaway cert keep the whole thing inside tmp_path and leave no state behind.
- Cost: the stack has to detect readiness from logs, which is the fragile part of the fixture.

What the fixture then proves

- An executable task runs and its stdout comes back (after the get_task re-fetch).
- A bash loop's output tails incrementally — log_offset advances, so it is not refetched whole.
- A job exiting 3 lands FAILED with a non-empty error.
- A sleeping job cancels while running, and reports CANCELED.